

R&D-7. Эмбединги как adjustment set в нерандомизированном испытании

Статус: базовый эксперимент готов. Реализованы тулkit-адаптеры (pyspark) и causal-слой (numpy/sklearn): оценка ATE с поправкой, диагностика баланса/overlap. Источник чисел — notebook.ipynb (синтетика, seed=11).

Цель

Проверить, могут ли клиентские эмбединги (в сокращённом виде) выступать как хотя бы сколько-то хороший adjustment set в Нерандомизированном испытании — то есть обеспечивать достаточный контроль смещения отбора при оценке причинного эффекта без рандомизации назначения воздействия

Данные

Исходный pyspark-датасет эмбедингов имеет схему:

epk_id, report_dt (месячная гранулярность), col_000, col_001, ..., col_{k}

Назначение воздействия хранится отдельно — датасет трита с колонками epk_id, report_dt, treatment (бинарный).

Тулkit-адаптеры (готово)

Реализованы в пакете src/rnd_reports/embeddings/ как «переходники» данных:

1. EmbeddingReducer (reducer.py) — снижает размерность эмбедингов до reduced_shape (по умолчанию 5) методом PCA (pyspark.ml). Выход: [epk_id, report_dt, emb_000, ..., emb_{reduced_shape-1}].

2. PropensityScorer (propensity.py) — джойнит эмбединги с датасетом трита по (epk_id, report_dt) и обучает LogisticRegression на сырых col_*, сводя их к единственному propensity_score = P(treatment=1). Выход: [epk_id, report_dt, propensity_score].

Оба адаптера поддерживают in-time safety: при заданном cutoff обучение идёт только на report_dt <= cutoff, а применение — к более поздним срезам (без утечки из будущего; ср. подход R&D-6 «safe in-time CUPAC»).

Контракты схем и валидация — contracts.py; конфиг — configs/07_embedding_adjustment_set/adapters.yaml.

Causal-слой (реализован)

src/rnd_reports/embeddings/experiment.py (numpy/sklearn, без pyspark — запускается в base-окружении):

- estimate_ate_with_adjustment(method=...) — ATE наблюдательных данных: naive (разность средних), propensity_weighting (стабилизированный IPW, propensity = LogisticRegression на эмбедингах), doubly_robust (AIPW: per-arm регрессия исхода + IPW-коррекция);
- covariate_balance_after_adjustment — |SMD| ковариат до/после IPW-взвешивания;
- overlap_diagnostics — overlap/positivity по распределению propensity;
- evaluate_adjustment_set_quality — сводка: ATE тремя методами + смещения vs эталон.

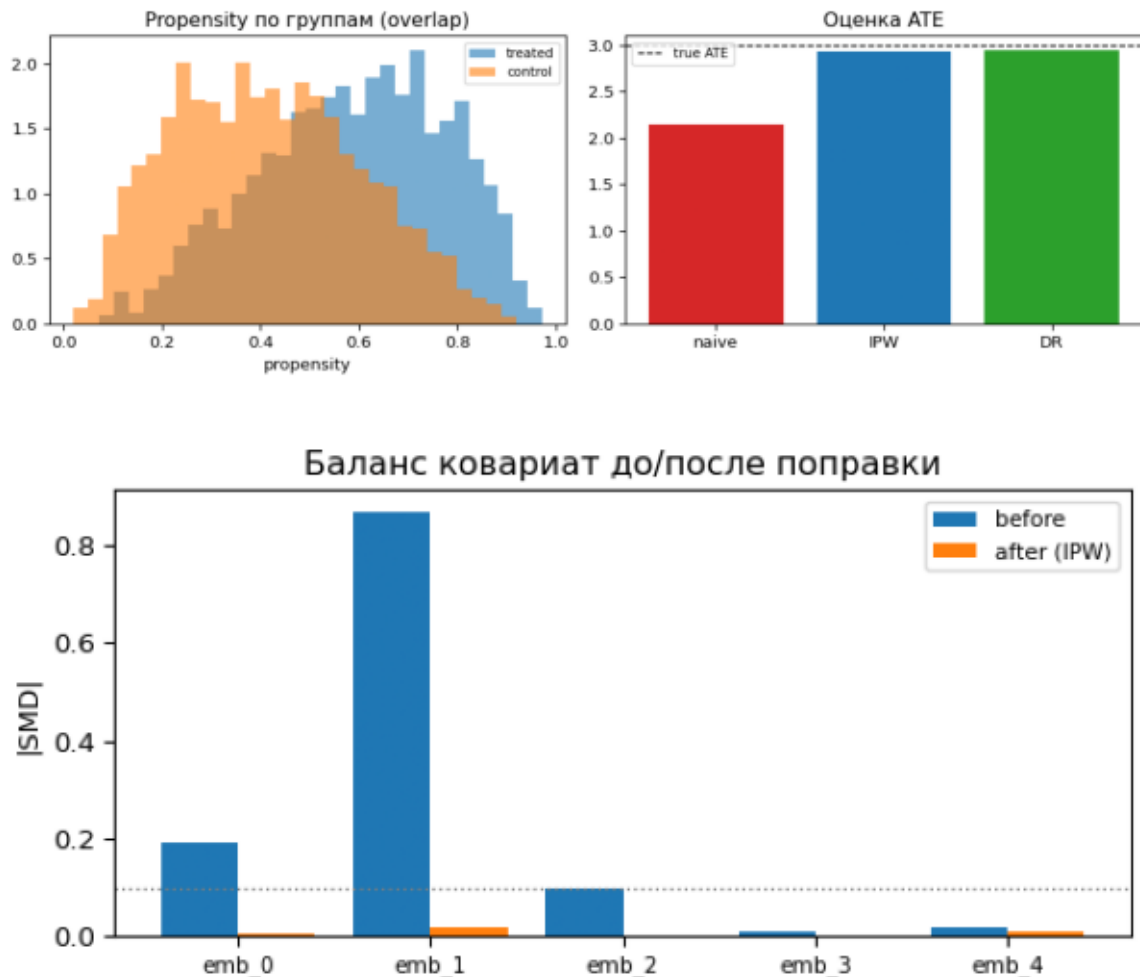
Синтетика — embeddings/synthetic.py (make_embedding_observational_scenario): латентные конфаундеры → эмбединги col_*; treatment ~ Bernoulli(sigmoid(confounding·z·a)) (селекция, не рандом); outcome = true_ate·T + z·b + ε. ATE известен по построению.

Результаты

Синтетика $n=4000$, $k=8$ эмбедингов $\rightarrow$ 5 PCA-компонент как adjustment set, истинный ATE = 3.0, seed=11.

метод	ATE	смещение vs true	снижение	SMD	/смещения
naive (без поправки)	2.145	-0.855	—		
IPW (propensity на эмбедингах)	2.930	-0.070	-91.8% смещения		
doubly_robust (AIPW)	2.948	-0.052	-94.0% смещения		

Баланс ковариат: $\max |SMD|$ 0.869 $\rightarrow$ 0.019 после IPW-взвешивания. Overlap: 97.6% propensity в [0.1, 0.9] (positivity выполняется).



Выводы

- Наивная разность средних смещена из-за селекции (трит коррелирует с конфаундерами через эмбединги).
- Сокращённые эмбединги как adjustment set + IPW/doubly-robust убирают ~92-94% смещения и восстанавливают баланс ковариат ($\max |SMD| \rightarrow \sim 0.02$) при хорошем overlap.
- Вывод: клиентские эмбединги несут достаточно информации о конфаундерах, чтобы служить практичным adjustment set в наблюдательном испытании — при условии overlap/positivity.
- Ограничения: результат на синтетике с известным ATE; на реальных данных нужны проверка overlap и sensitivity к неучтённым конфаундерам (unconfoundedness напрямую не тестируема). Production-путь — pyspark-адаптеры EmbeddingReducer/PropensityScorer с in-time safety (обучение на $\text{report_dt} \leq \text{cutoff}$).

Воспроизведение

```
pip install -e .          # base (numpy/pandas/scipy/sklearn) – эксперимент запускается без pyspark
pip install -e .[spark]   # опц.: pyspark для production-адаптеров и их тестов
python tools/execute_notebooks.py  # выполнить notebook.ipynb (→ results/07.../figures/)
python tools/generate_pdf.py       # пересобрать report.pdf
```